Domain-Specific Search: BM25 vs. Neural Retrieval on Technical Documents

Intermediate Status Update | Information Retrieval | Group 10 | 25 October 2026

Surajsinh Patil 2025H1030058P Tejas Mahajan 2025H1030057P
Swayam Kumud 2025H1030063P Saksham Singhal 2025H1030208P

Repository: https://github.com/TODO/bm25-vs-neural

1 Implementation status and architecture

We compare our own BM25 implementation with two dense retrievers used without fine-tuning (E5-base-v2 and SPECTER2) and their Reciprocal Rank Fusion hybrids on a frozen corpus of 39,613 arXiv cs.CR abstracts, using 50 queries in five categories (exact identifier, homonym, negation, time constraint, paraphrase). The whole pipeline is implemented, unit-tested and runs end to end; Table 1 lists the components.

Table 1: Components built so far versus pending. Every built component has unit tests (23 tests: BM25 against the formula, metrics against `pytrec_eval`, kappa against scikit-learn, fusion, pooling).

Component	Status	Notes
Corpus builder (`corpus.py`)	done, frozen	39,613 docs, SHA-256 `fda391ecc5b6`; arXiv id list committed
Identifier-aware tokeniser + BM25	done, validated	SciPy sparse matrix, vectorised scoring; matches Lucene (Sec. 3)
Dense encoders + exact search	done	E5 (mean pooling, prefixes), SPECTER2 (proximity / ad-hoc query adapters)
RRF hybrids (k=60)	done	BM25+E5, BM25+SPECTER2 over top-100
Metrics, significance, CIs	done, validated	nDCG, P, MRR, MAP, R, Hole@10; permutation test + Holm; bootstrap CIs
Query set (50 + 5 practice)	done	TREC format; 10 anchors confirmed in corpus, 40 being confirmed
Pooling + blind judging tool	done	1,339 pairs (≈26.8/query), 2 judges/query, offline HTML page
All five runs on cs.CR and SciFact	done	Kaggle T4 GPU; TREC run files and time/memory logs in the repository
Relevance judging	**in progress**	pages generated; phase 1 (any run's top-5, 722 pairs) first [update]
Evaluation on cs.CR, k1/b sweep, ablation	code done	run end to end; waiting for the frozen qrels
Stretch S1, S2	runs done	cross-encoder and date-filtered runs produced; their new documents need judging
Stretch S3, S4	code done	LLM judge, CC fusion (need qrels)

Changes from the proposal. (1) The Pyserini check builds a Lucene index from exactly the same title+abstract text, so the two BM25s differ only in the implementation. (2) E5 is run through `transformers` with our own mean pooling, because `sentence-transformers` and the `adapters` library needed by SPECTER2 require incompatible library versions; a test checks the pooling against a manual computation. (3) Besides validating BM25 on SciFact we run *all five* systems there, which gives judged results before our own judgments are finished. (4) A judgment-free analysis of top-10 overlap between systems was added. (5) For S2 we first use a rule-based date extractor (no API cost); the LLM extractor is optional. (6) The filtered collection has fewer than the planned 50,000 documents, so it is used in full (no sampling). Figure 1 shows the updated architecture.

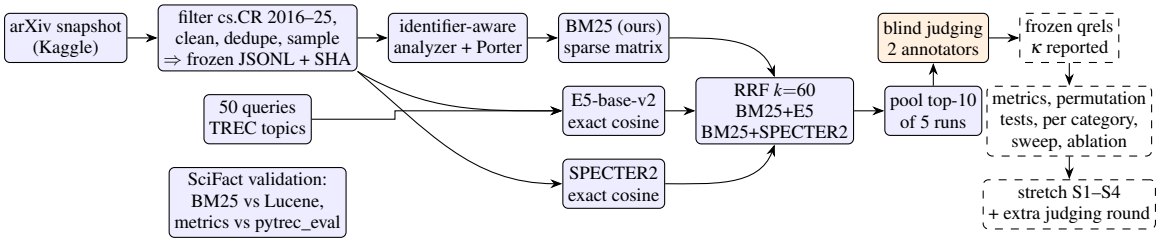


Figure 1: Updated architecture. Shaded = implemented and run; orange = in progress; dashed = code written and tested, waiting for the frozen qrels. Each stage is frozen before the next starts.

2 IR pipeline details

Corpus and preprocessing. From the Kaggle arXiv snapshot we keep papers with cs.CR as primary or cross-listed category and a first version in 2016–2025, remove withdrawn papers (35) and duplicate titles (12), strip LaTeX and apply NFKC normalisation. Of 53,454 cs.CR records in the snapshot, 39,613 remain and all are used (26,487 with cs.CR as primary category). A document is its title plus abstract.

BM25 (lexical). The identifier-aware analyzer lower-cases, joins protocol versions ("TLS v1.3" → `tls1.3`), keeps identifiers such as `cve-2021-44228`, `sha-256` or `x86-64` as single unstemmed tokens and also indexes their parts, which do not count towards document length (like Lucene's overlapping tokens). Other words pass through Lucene's 33 English stopwords and the Porter stemmer. Note that "no" and "not" are stopwords, so BM25 cannot see negation at all. Term frequencies are stored

Table 2: BEIR Scifact test set (300 queries, 5,183 abstracts); BM25 with k_1=0.9, b=0.4.

System	nDCG@10	P@5	MRR@10	MAP@10	R@10	ms/query
Pyserini BM25	0.679	0.160	0.646	0.633	0.804	62.7
BM25 (plain tok.)	0.678	0.160	0.643	0.632	0.804	1.1
BM25	0.677	0.160	0.643	0.631	0.803	1.2
E5	0.727	**0.175**	0.693	0.683	0.851	1.8
SPECTER2	0.646	0.156	0.610	0.600	0.774	2.3
BM25+E5	**0.741**	**0.175**	**0.711**	**0.696**	**0.864**	–
BM25+SPECTER2	0.717	0.169	0.676	0.667	0.861	–

Table 3: Paired randomisation tests on SciFact nDCG@10 (300 queries, 100,000 permutations, Holm over all 10 pairs).

Comparison on SciFact	ΔnDCG@10	p_{Holm}	Sig.
E5 − BM25	+0.051	0.0235	yes
SPECTER2 − BM25	-0.031	0.3455	no
BM25+E5 − BM25	+0.065	0.0001	yes
BM25+SPECTER2 − BM25	+0.041	0.0126	yes
SPECTER2 − E5	-0.082	0.0001	yes
BM25+E5 − E5	+0.014	0.4953	no
BM25+SPECTER2 − SPECTER2	+0.072	0.0001	yes

once in a SciPy CSC matrix (56,862 terms); for given (k_1, b) the whole BM25-weighted matrix is computed in one vectorised step with the Lucene IDF $\ln(1 + \frac{N-n_t+0.5}{n_t+0.5})$, and a query is scored with one sparse matrix–vector product. The main run uses k_1=1.2, b=0.75.

Dense retrieval. Both models are BERT-base (768 dimensions), used as published. E5 uses the `query:/passage:` prefixes and mean pooling; SPECTER2 encodes "title [SEP] abstract" with its proximity adapter and queries with its ad-hoc query adapter ([CLS] vector). Documents are sorted by length for batching (fp16 on a Tesla T4). Truncation is not an issue: arXiv limits abstracts to 1,920 characters, the median document has 261 tokens and fewer than 0.1% exceed the 512-token limit. Embeddings are cached and searched exactly (brute-force cosine), so no ANN approximation error enters the comparison. **Hybrids** use RRF (k=60, fixed beforehand) over the top-100 of each run, so no score normalisation and no tuning on our queries is needed.

Judging and evaluation. The top-10 of all five runs is pooled. Each query goes to two of the four annotators (25 queries, $\approx$750 judgments each, every annotator pair shares queries so κ covers all pairs). The judging page shows documents in random order with no system or rank, shows dates only for time queries, saves progress in the browser and exports JSON. Disagreements are listed for an adjudication meeting and the qrels are then frozen with a checksum. Metrics follow `trec_eval` conventions (ties broken by docid, unjudged = non-relevant, binary metrics at grade ≥ 2). Significance: paired randomisation test (100k permutations, exact enumeration for $n \leq 16$) with Holm–Bonferroni; per-category means get 95% bootstrap CIs.

3 Preliminary evaluation results

Validation (success criteria i and ii). On BEIR SciFact with identical settings (k_1=0.9, b=0.4, Lucene-like analyzer) our BM25 differs from Pyserini by ΔnDCG@10 = -0.0009 (criterion i passes, threshold 0.01) and shares 98.0% of its top-10 documents. Our metric code reproduces `pytrec_eval` on every run and query (criterion ii passes). The same check on an offline 10,788-document collection gave ΔnDCG@10 = -0.0026, 96.8% top-10 overlap, and a score ratio of 1.900 to Lucene, i.e. exactly the constant (k_1+1) that Lucene omits; the small remaining differences come from Lucene's lossy encoding of document length.

First comparison against the BM25 baseline (SciFact, 300 queries). E5 beats BM25 by +0.050 nDCG@10 (p_{Holm} = 0.023; better on 82 queries, worse on 46), while SPECTER2 is 0.031 *below* BM25 (not significant, p_{Holm} = 0.346) and clearly below E5 (p_{Holm} <0.001). BM25+E5 is the best system (0.741), significantly better than BM25 but not than E5 alone (+0.014, p_{Holm} = 0.495); its main effect is robustness: it is worse than BM25 on only 18 queries versus 46 for E5. BM25+SPECTER2 is significantly better than both of its components. So on SciFact the answers are RQ1 yes for E5 only, RQ3 no (the general model wins), RQ4 partly. SciFact queries are natural-language claims, which favours dense models; our identifier, negation and time queries test whether this holds on cs.CR.

cs.CR collection, no judgments needed. Table 4 shows that the systems retrieve very different documents. BM25 and E5 share 0.35 of their top-10 on average, from 0.51 on homonym and 0.44 on identifier queries down to 0.13 on paraphrases; SPECTER2 overlaps little with either. The near-disjoint lists on paraphrases are what we expected, and they mean the hybrid always has complementary candidates. For the ten time queries we can already count, from the first-version date, how many top-10 results break the date condition: 38% for BM25, 32% for E5, 30% for SPECTER2, 35% for both hybrids and 31% after cross-encoder re-ranking (S1). No system uses the condition; the S2 date filter brings this to 0%. Whether that raises nDCG depends on the remaining documents being relevant, which judging will show. Efficiency (Table 5): BM25 indexes the corpus in 20.3 s on one CPU core into 29.3 MB and answers a query in 1.6 ms. Each dense index is 116 MB (4× larger)

Table 4: Mean overlap of the top-10 lists (fraction of shared documents), no judgments needed.

Pair	Exact	Homonym	Negation	Time	Paraphrase	All
BM25 / E5	0.44	0.51	0.27	0.41	0.13	0.35
BM25 / SPECTER2	0.24	0.25	0.18	0.26	0.10	0.21
E5 / SPECTER2	0.26	0.26	0.22	0.38	0.12	0.25

Table 5: Efficiency on the cs.CR corpus. Dense indexing = encoding the corpus on the GPU; dense query time includes encoding the query; hybrid = sum of its components (p95 not defined).

System	Index (s)	Index (MB)	Query (ms)	p95 (ms)	Hardware
BM25	20.3	29.3	1.6	1.7	cpu
E5	174.6	116.0	17.3	18.2	Tesla T4
SPECTER2	182.6	116.0	30.0	36.6	Tesla T4
BM25+E5	194.9	145.4	19.3	–	–
BM25+SPECTER2	202.8	145.4	32.0	–	–

and takes about 3 minutes of T4 time to build. A dense query takes 17–30 ms, mostly query encoding (exact search over all vectors takes about 6 ms). Cross-encoder re-ranking of 100 documents adds 456 ms per query.

4 Path to final submission

Table 6: Plan for the remaining work (final submission mid-November).

Dates	Work
26–31 Oct	Finish judging (phase 2: ranks 6–10); compute κ (target ≥ 0.6, otherwise revise guidelines and redo affected queries); adjudication meeting; freeze qrels.
1–4 Nov	Main results (RQ1, RQ3, RQ4) with significance tests; per-category analysis with CIs (RQ2); k1/b sweep with 5-fold CV and tokenisation ablation (RQ5); check our pre-registered expectations.
5–9 Nov	Stretch goals S1 (cross-encoder re-ranking of the best run's top-100) and S2 (date filter on time queries); judge their new documents in an extra blind round.
9–12 Nov	Error analysis and case studies per category; S3/S4 if time allows.
12–15 Nov	Final report, figures, release code, queries, qrels and runs; demo preparation.

Stretch objectives. S1 and S2 have been run on cs.CR; S3 and S4 are implemented and tested on synthetic data. **S1** re-ranks the top-100 of BM25+E5 with `ms-marco-MiniLM-L-6-v2` (run done); we expect gains mainly on negation queries. **S2** parses the date range of each time query ("after its 2018 standardisation" $\rightarrow \geq$2019; all 10 parsed correctly) and filters on the first-version date (run done, removes 3.5 of the top-10 on average). The new documents these runs bring into the top-10 will be judged blind in a second round once the main qrels are frozen. **S3** grades the same pool with an LLM through any OpenAI-compatible endpoint and reports κ and Kendall's τ of the system ranking. **S4** tunes a convex combination of min-max normalised scores with stratified 5-fold CV. Order of priority is unchanged: S1, S2, then S3/S4. **Risks.** Judging time (mitigated by phase-1 ordering); queries without a confirmed relevant document (replaced via random abstracts, never via our systems); possible training-data overlap of the dense models with arXiv abstracts (reported as a limitation).

5 Contribution matrix

Member	Components owned and completed to date
Surajsinh Patil	[e.g. BM25 implementation, identifier-aware tokeniser, validation against Pyserini]
Tejas Mahajan	[e.g. corpus builder and query set]
Swayam Kumud	[e.g. dense encoders, GPU encoding, RRF]
Saksham Singhal	[e.g. evaluation, significance tests, judging tool]
All	Query writing and review, relevance judging (two annotators per query)